RAZVOJ VISESLOJNIH APLIKACIJA U EES

1. Procesi razvoja softvera
    - Kaskadni proces
    - Iterativni (fazni proces)
2. **Kaskadni (waterfall) postupak ***
    - Analiza softvera
    - Projektovanje
    - Programiranje
    - Testiranje
    - Isporuka i odrzavanje

    Za svaku od gore navedenih softvera se definise odredjeni vremenski period.
3. **Iterativni (fazni) postupak ***

    Deli projekat na podskupove po funkciji, na primer jednogodisnji projekat se deli na tromesecne iteracije. Svaki kvartal sadrzi sve faze razvoja kaskadnog procesa. Postoji jedan dodatni deo, istrazivanje, koje prethodi samom procesu. U svakoj od faza se stvara privid funkcionalnosti softvera
4. **Problemi u kaskadnim procesima ***
    - Cesti povratni tokovi (povratak na analizu...) u slucaju da se dodaju novi zahtevi ili utvrdi da softver nije dovoljno kvalitetan
    - Posto su sve faze jasno definisane kada prodjemo neku fazu ne bi smeli da se vracamo na nju, problem nastaje prilikom utvrdjivanja da li je faza apsolutno zavrsena/korektna.
5. **Problemi u faznim procesima ***
    - Nastaje problem prilikom “usaglasavanja” sa predifinisanim planom faza (neka faza moze da se zavrsi ranije a neka kasnije). Ukoliko se radi na velikim projektima i vremenski rok faza se ne definise kako treba to moze dovesti do nezadovoljstva radnik koji bi u nekim delovima bili preotereceni a u nekim ne bi imali sta da rade. Istrazivacka faza je veoma bitna.
6. Prednosti u faznim procesim
    - Delovi softvera koji se u nekoj vazi razviju mogu da se isporuce (isporucivanje po fazama - Releases)
7. Hibridni procesi
    - Hibridni procesi predstavljaju kombinacije kaskadnih i iterativnih procesa. Neke od faza se odvijaju kaskadno a neke iterativno
8. Inkrementalni model

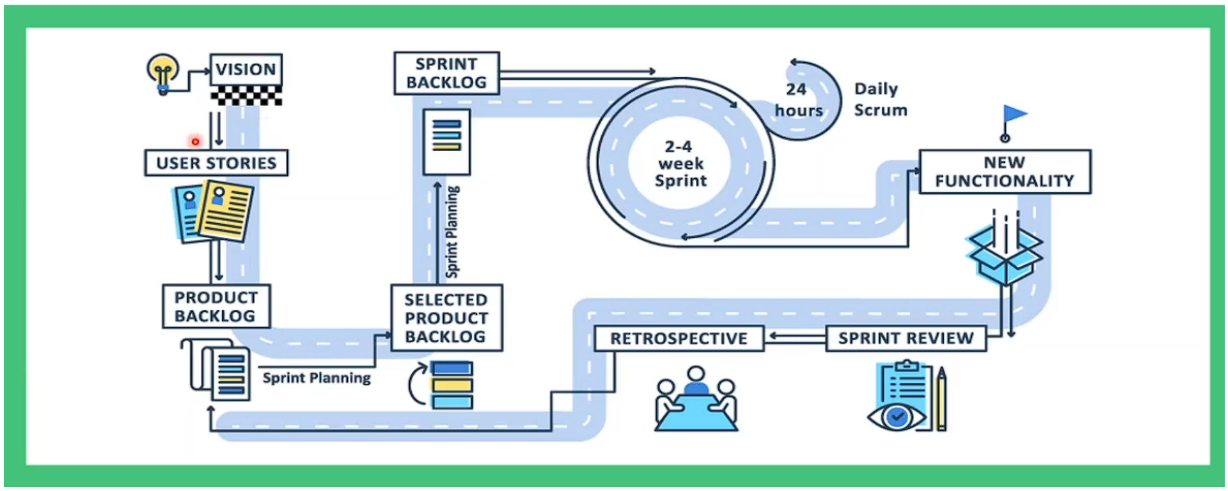
    Podrazumeva da korisniku omogucimo da vidi samo onaj deo koji je realizovan i funkcionalan.
9. Tehnike za povecanje efikasnosti razvoja
    - Automatski regresivni testovi (Unit testovi)
    - Refaktorisanje koda (promena postojeceg koda koja ne utice na ponasanje)
    - Neprekidna integracija (postupak za sinhronizaciju tima, merge)

crvenom bojom i * su oznacena pitanja koja sigurno dolaze na usmeni

RAZVOJ VISESLOJNIH APLIKACIJA U EES

10. SCRUM proces

    Agilni procesi nam omogucavaju da ostanemo u toku sa dinamickim (konstantno se dodaju ili menjaju neke funkcionalnosti) razvojem nekog softvera.



- Vizija (Vision) predstavlja sam plan/viziju kako bi softver trebao da izgleda, njegove funkcionalnosti, kakve potrebe softver treba da zadovoljava itd.
- User stories, predstavlja zadatke, fukcionalnosti koje softver treba da zadovolji. Spisak opisa funkcionalnosti.
- Product backlog, zadaci se pamte i cuvaju.
- Sprint planning, planiranje “sprinta”, izdvajanje zadataka koji ce se u tom “sprintu” razvijati, potom planiranja ti zadaci se smestaju Selected product backlog.
- Potom sledi jos jedna faza planiranja gde se zadatak rastavlja na delove i tacno utvrdjuje sta ce biti zavrseno u toku sprinta.
- Sprint backlog, sadrzi delove zadatka koji treba da se implementiraju u toku sprinta.
- Svakodnevno se organizuje Daily Scrum, u toku kojeg svaki clan tima objasniti sta je radio/uradio u toku prethodnog dana.
- Posle 2-4 nedelje, implementirane funkcionalnosti se komituju/merge-uju i na taj nacin dobijamo novu funkcionalnost (New Functionality).
- Sprint Review, predstavlja sastanak gde se utvrdjuje sta je od planiranih stvari uradjeno, sta se nije moglo uraditi, zasto itd…
- Retrospective, sta je to bilo dobro u toku prethodnog sprinta, sta nije bilo dobro, odnosi u timu itd...

11. SCRUM uloge

    Proces podrazumeva da clanovi tima/firme imaju vise uloga.

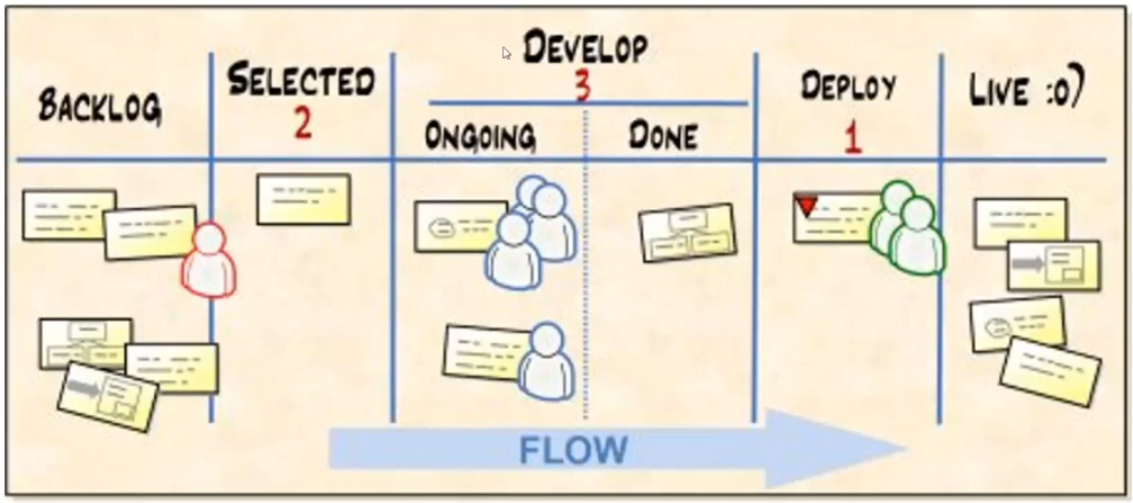
- Scrum master, osoba koja organizuje SCRUM sastanke, organizuje/vodi jedan tim.
- Clanovi tima (Developeri/Testeri), implementirati i testiraju funkcionalnost.
- Stakeholderi, svi oni koji su zainteresovani za sam proizvod, korisnici, menadzeri.
- Product Owner, osoba koja direktno komunicira sa korisnicima i zaduzena je za isporuku i dogovor oko zahteva.
- Analiticari, analiza sta je moguce implementirati.

crvenom bojom i * su oznacena pitanja koja sigurno dolaze na usmeni

RAZVOJ VISESLOJNIH APLIKACIJA U EES

12. KANBAN

    U svakoj koloni postoje neke kartice sa infomacija o svakoj fazi, o tome do koje faze je odredjena funkcionalnost dosla/se nalazi u. Moguce je voditi vise projekata paralelno i cak osobe mogu da rade na vise projekata istovremeno.



    - Backlog, deo u koji se dodaju zadaci koji pokrivaju odredjene funkcionalnosti.
    - Selected, izvuceni zadaci koji treba da se implementiraju.
    - Develop, faza za razvijanje samog zadatka, dve pod faze ongoing i done.
    - Deploy, ako je nesto odradjeno salje se na koriscenje.
    - Live, ukoliko se nesto koristi postavlja se u live fazu.

13. SCRUM vs KANBAN
    - Scrum je mnogo slozeniji od kanbana i zahteva mnogu studiozniju pripremu.
    - Scrum vrsi isporuku na kraju sprinta, dok je kod kanbana neprekidna isporuka.
    - Kod kanbana-a nema novih uloga, tim i proces postuju uloge u kojim se zateknu.
    - Kod sprinta promena nije moguca dok kod kanbana promena je moguca u bilo kom trenutku.

14. Gledista u pogledu razvoja softvera
    - Konceptualno, ne uzima u obzir softver i realizaciju, objekat je skup aktivnosti
    - Specifikaciono, aspekat korisnika a ne realizacije, objekat je skup metoda koje pozivaju drugi objekat ili taj objekat.
    - Realizaciono, posmatra se kod, najcesce koriscena perspektiva. Objekat predstavlja njegov kod i podaci.

15. UML (Unified Modeling language) *

Predstavlja objektni jezik za modelovanje sacinjen od skupa grafickih, standardizovanih oznaka. Koristi se za reprezentaciju programa (veze izmedju objekata). Kada se opisuje

celokupan sistem UML mozemo smatrati kao izvorni (source) kod. Vecina razvojnih okruzenja moze neposredno da prevede UML u izvrsni kod.

- Vizuelizaciju
- Specifikovanje
- Konstruisanje
- Dokumentaciju

16. Razlozi koriscenja UML-a *

- Komunikacija
- Bolje razumevanja samog softvera/sistema
- Preciznost (dobar opis koda sa aspekta OOP)
- Kvalitetna dokumentacija

17. Izrada skica/modela

- Direktan razvoj (forward engineering), model se izradjuje pre pisanja samog koda. Lakse saostavanje ideja i brza analiza koda ciji razvoj dugo traje su glavne prednosti ovog nacina formiranja modela.
- Povratna analiza (reverse engineering), model se pravi prema vec postojecem kodu/softveru. Sluzi za bolje razumevanje koda, najcesce samo zanimljivih delova koda.

18. Notacija i Metamodel

- Notacija predstavlja sintaksu UML-a, jezika za modelovanje (definise kako se odredjeni elementi/things predstavljaju).
- Metamodel definise koncepte jezika/UML-a u ovom slucaju.

19. Tipovi UML dijagrama koji se najcesce koriste *

- Dijagram klasa (klase, veze izmedju njih, klasni atributi)
- Dijagram interakcije (kombinacija dijagrama sekvenci i aktivnosti)
- Use Case dijagrami (interakcija korisnika i sistema)
- Dijagram aktivnosti (proceduralno i paralelno ponasanje)
- Dijagram stanja (kako dogadjaji menjaju objekat)
- Dijagram deployment-a (fizicka organizacija sistema)

20. Blokovi koji se koriste za izgradnju UML dijagrama *

- Stvari/Elementi (strukturni, statusni, grupisuci, anotacijski)
- Relacije (zavisnosti, asocijacije, generalizacije, realizacije)
- Dijagrami

21. UML klasa *

Klasa predstavlja skup objekata koji deli iste atribute, operacije, relacije i semantiku.

- Graficki se predstavljaju pravougaonikom

- Ime klase je predstavjeno nizom znakova i uvek pocinje velikim slovo, ukoliko ima vise reci koristi se kamilja notacija
- Moze postojati vise atributa u jednoj klasi, atributi predstavljaju imenovanu osobinu klase. Pocinju malim slovom i ukoliko ima vise reci svaka sledeca pocinje velikim.
- Operacija predstavlja implementaciju nekog servisa (apstrakcija akcije nad objektom). Pocinje malim slovom, u slucaju vise reci koristi se kamilja notacija.

22. UML relacije *

- Zavisnost (relacija koriscenja)
- Generalizacija (povezuje opstiju klasu sa specificnom natklasom/potklasom)
- Asocijacija (strukturna relacija izmedju konkretnih primeraka)
- Realizacija (relacija izmedju interfejsa i klase koja ga implementira)

23. Relacija zavisnosti *

Relacija koja oznacava da promena u specifikaciji jedne stvari moze uticati na drugu stvar. Najcesce se koristi kao prikaz da jedna klasa koristi drugu kao argument u deklaraciji operacije. Oznacava se isprekidanom linijom. Dodatno se opisuje pomocu kljucnih reci (<< >>, neke od kljucnih reci su call, create, derive, permit).

24. Relacija generalizacije *

Odnos potklasa/natklasa, potklase uglavnom imaju svoje atribute.

25. Relacija asocijacije *

Povezuje dve klase. Opciono asocijaciju mozemo detaljnije opisati nekim imenom (“radi za”), usmerenjem, ulogom, grupisanjem, oznakom visestrukosti. Moguca je dvosmerna asocijacija (oznava se sa strelicom na oba kraja), kada postoji par svojstava iz razlicitih klasa koja su povezana (osoba poseduje automobil, automobil je u vlasnistvu osobe...valjda je to to).

26. Vidljivost dijagrama klasa
    - Javni (public) +
    - Privatni (private) -
    - Paketni (package/namespace) ~
    - Zasticeni (protected) #
27. Kardinalnost (multiplicity)

    Na koliko objekata se odnosi neko svojstvo.
    - 1 (jedna porudzbina ima tacno jednog kupca)
    - 0..1 (ne mora ni jednog ali maskimalno jednog)
    - * (ne mora ni jednog ali moze koliko god)
28. Rezervisane reci

    Pojasnjavaju znacenje nekih simbola. Pisu se uz pomoc {} i << >>.
29. Odgovornosti

    Kratke recenice koje opisuje znacenje funkcija klase.

30. Agregacija i kompozicija

    Odnose se na predstavu grupisanja nekih elemenata.

- Agregacija predstavlja odnos celina-deo (na primer automobil-tockovi, oznacava se sa praznim rombom.
- Kompozicija predstavlja odnos koji pokazuje da jedna instanca ciljanje klase moze da pripada samo jednom vlasniku, oznacava se sa punim rombom.

31. Izvedena svojstva
    Mogu se izrazunati na osnovu drugih svojstava, oznacavaju se / crtom.
32. Aktivna klasa
    Ima instance od kojih svaka izvrsava svoju programsku nit i upravlja njom. Oznacavaju se dodatno prosirenim pravougaonikom sa strane.
33. **Dijagrami sekvence ***
    Formiraju se pre samog razvoj softvera (najcesce) u cilju jasnog dokumentovanja sta se ocekuje od najvaznijih metoda/procedura nekog softvera. Spadaju u dijagrame ponasanja. Grupa u koju se svrstavaju je grupa interakcije. Ono sto dijagrami sekvence omogucavaju je da se opise jedan scenario koji obuhvata razmenu poruka izmedju vise objekata sa ciljem da se opise neki slucaj koriscenja. Opisuju interakciju izmedju vise ucesnika, ucesnici su prikazani kao neki objekti koji imaju svoj zivotni ciklus/vek (opisani su vertikalnim linijama).
34. **Elementi dijagrama sekvenci ***
    - Ucesnici/Objekti
    - Poruke koje se razmenjuju
    - Linija zivota (opisuje koliko je neki ucesnik aktivan u samoj aplikaciji)
    - Traka aktivnosti (koliko je ucesnik aktivan u izvrsavanju odredjenog koraka)
    - Primljena poruka (zahtev koji se analizira)
35. **Dizajn/arhitektura dijagrama sekvence ***
    - Centralno upravljanje (klasa vrsi komunikaciju sa drugim klasam, poziva druge metode itd…)
    - Distribuirano upravljanje (potrebno je da klasa ima sto manje odgovornosti, svaka klasa je preodredjena za samo jedan posao, klase su specijalizovane...ukoliko na postojeci softver treba da dodamo nove funkcionalnosti distribuirano upravljanje je bolje jer je samo odrzavanje/dodavanje mnogo jednostavnije.
36. **Mogucnosti dijagrama sekvenci ***
    Jasno naznaka da neka klasa kreira neki drugi objekat (pomocu strelice sa porukom new), brisanje se oznacava sa simbolom X. Postoji mogucnost prikazivanja petlji i uslova medjutim one nisu bas najbolje vidljive. Petlje i uslovi se prikazuju unutar okvira interakcije (interaction frames), sadrze jedan ili vise fragmenata koji se u odredjenim uslovima mogu izvrsavati, takodje u svom zaglavlju sadrze informaciju na koji nacin se petlja ponavlja. Starije konvencije su koristile simbole zvezdice (*) kako bi oznacavali pocetak frejmova iteracije dok su se uglaste zagrade ([ ]) koristile za prikazivanje uslova. Strelce sa lopticom su ukazivale na kretanje podataka.

37. Operatori okvira interakcije
    - Par, paralelno izvrsavanje svih fragmenata
    - Region, fragment koji govori da ne moze imati istovremeno izvrsavanje vise niti

- Neg, pokazuje pogresnu interakciju
- Ref, referenca na interakciju drugog dijagrama
- Sd, oznaka za ceo dijagram, fragment dijagrama sekvence

38. Sinhrone i Asinhrone poruke kod sekvencijalnih dijagrama
    Popunjen vrh strelice predstavlja sinhronu poruku dok prazan vrh oznacava asinhronu. Poruka takodje moze biti povratna ili izgubljena (lost/found, koristi se kada nam za samu analizu nije bitno gde je poruka otisla, kao i za poruke koje dodju od nekuda).
39. Kada se koristi dijagram sekvence
    - Kada zelimo da analiziramo komunikaciju izmedju objekata
    - Kada imamo potrebu da resavamo probleme vezane za deadlock
40. CRC kartice
    CRC(Class-Responsibility-Collaboration), govore o nadleznosti svake klase, svaka klasa se opisuje preko svog malog skupa odgovornosti. Pomocu CRC kartica se razvijaju razliciti scenariji, kartice se premestaju/podizu cime se formira slanje/prosledjivanje podataka (sekvencijalni dijagram).
41. **Dijagrami ponasanja ***
    Predstavljaju dijagrame koji opisuju kako nasa aplikacija radi. U njih spadaju dijagrami aktivnosti, slucaja upotrebe, stanja i interakcije (grupa dijagrama medju kojima je najpoznatiji sekvencijalni dijagram medjutim nalaze se i dijagrami komunikacije, interakcije i vremenski dijagrami).
42. **Dijagram slucaja upotrebe ***
    Vezani su za situaciju u kojoj analiziramo zahteve za softver koji razvijamo. Funkcionalni zahtevi softvera su kreirani od strane krajnjeg korisnika. Predstavlja nacin interakcije korisnika i sistema. Sastoji se iz skupa scenarija (use case) koji je povezan jednim ciljem korisnika. Scenario predstavlja niz koraka koji predstavljaju interakciju korisnika i sistema/softvera. Nisu mnogo dokumentovani i jako je vazno tekstualno obraditi slucajeve. Predstavljaju spoljni pogled na sistem i ne opisuju klase sistema. Dijagrami prikazuju:
    - Ucesnike (uloge-role, terminoloski je pravilnije), predstavljaju ulogu korisnika u odnosu na sistem/softver...da li je on kupac, prodavac, menadzer, administrator nije bitno.
    - Slucajeve upotrebe
    - Veze izmedju slucajeva upotrebe

43. **Sadrzaj slucaja upotrebe ***
    Na pocetku se bira jedan osnovni uspesni scenario (njegovi koraci), kasnije se dodaju drugi scenariji kao prosirenja. Definise se glavni ucesnik (obicno inicijator) i ostali

(sporedni ucesnici). Korak predstavlja nameru ucesnika, za svaki korak je potrebno analizirati sve povoljne i nepovoljne dogadjaje. U slozenom slucaju upotrebe jedan slucaj upotrebe moze biti korak u drugom (koristi se <<include>> zavisnost). U slucajve upotrebe se mogu dodati:

- Preduslov (pre-condition), uslovi da pocne koriscenje
- Garancija (guarantee), stanje sistema na kraju upotrebe
- Okidac (trigger), dogadjaj koji inicira slucaj upotrebe.

44. Dijagrami stanja *

Vezani su za stanja objekta, ona mogu biti uslovljena nekim drugim stanjima. Pocetno pseudostanje se vezuje za nastanak objekta (da li postoji ili ne). Prelaz ukazuje na pomeranje iz jednog stanja u drugo, ima obelezje koje se sastoji iz tri dela:

- Potpis okidaca (dogadjaj koji omogucava prelaz u drugo stanje)
- Uslov (logika koja uslovljava da li moze da dodje do prelaza iz jednog stanja u drugo)
- Ponasanje (ponasanje koje se izvrsava tokom prelaza)

Ako se koristi vise prelaza za isti dogadjaj moraju biti medjusobno iskljucivi.
Zanemeren dogadjaj je onaj koji nije opisan nekim prelazom.
Zavrsno stanje je nesto sto zaustavlja proces prelazenja iz jedno u drugo stanje.
Prelazi se na kraju uglavnom prikazuju tabelarno, tabela sadrzi izvorno stanje, odredisno stanje, dogadjaj, uslov i aktivnost. Tabela stanja se napravi tako da za aplikaciju bude konfigurabilna (najcesce se pise u XML-u), obuhvata sva stanja i sve prelaze kao i dogadjaje, uslove i aktivnosti. Na taj nacin se aplikaciji omogucuje da radi univerzalno i da odgovori zahtevima svakog klijenta koji mogu biti zainteresovani za razlicite tokove stanja.

45. Obrazac-State (obrazac stanja)

Opisuje kako neka klasa, kontroler moze da se implementira za razlicitim stanjima u okviru neke aplikacije.

46. Dijagram aktivnosti *

Sluze za opisivanje toka logike (lice na dijagrame aktivnosti). Za razliku od dijagrama aktivnosti (flowcharts) mogu prikazati paralelno ponasanje/izvrsavanje. Polazi se od pocetnog cvora, izvrsava se akcija (ReceiveOrder) pa dolazi do grananja (paralelno izvrsavanje - redosled nije vazan.) Jasno je definisan (join), mesto gde se niti spajaju/sinhronizuju. Odluka (decision) ima jedan ulazni i nekoliko izlaznih tokova (u zavisnosti od uslova, koriste se [ ]). Stapanje (merge) im vise ulaznih tokova i jedan izlazni.

47. Razlaganje akcija u dijagramima aktivnosti *

Predstavlja izdvajanje pojedinih delova dijagrama u posebne akcije, ugnjezdenje u okviru glavnom dijagrama. Particije koje se dobiju razlaganjem pokazuju koja klasa ili celina izvrsava date akcije (odredjeni skup akcija potreban za neku celinu). Particija se cesto naziva i plivacka staza (swim line).

48. Signali u dijagramima aktivnosti *

Vremenski signali (dogadjaji koji se desavaju u nasem sistemu, posle nekog vremena ili u nekom trenutku). Postoje aktivnosti koje osluskuju neke signale. Signali se mogu

primati ali i slati. Signali mogu biti incirani od strane naseg sistema ali mogu biti i neki spoljni dogadjaji koji su inicirani od strane korisnika ili neke spoljne aplikacije.

49. Vrste dijagrama interakcije *

- Dijagrami komunikacije imaju zadatak da za neke proizvoljne ucesnike definisu komunikaciju u odredjenim situacijama, redosled razmena poruka. Malo lice na informacije koje nude sekvencijalni dijagrami. Sve poruke koje se salju ka ostalim objektima su definisane u redosledu (redosled se utrvrdjuje nekim naznakama, najcesce brojevima).
- Dijagram pregleda interakcija, predstavlja kombinaciju dijagrama aktivnosti i dijagrama sekvence. Retko se koristi.
- Vremenski dijagrami, postoji vise nacina prikazivanja. Uvode neka vremenska ogranicenja za neke operacije (primer aparata za kafu).

50. Dijagrami komponenata *

Komponente su neki delovi naseg softvera koje se mogu koristiti odvojeno/nezavisno od strane kupca softvera (moze da kupuje deo po deo). Komponente se prikazuju kao pravouganici koji u sebi sadrze pravougaonike. Podela na komponente je vezana za komercijalni deo (kako bi se sto lakse prodao softver i ostvario sto veci profit).

51. Dijagrami slozene strukture *

Kada imamo neki slozeni objekat (npr implementira i koristi razne intrefejse), moguce ga je prikazati kao klasu u kojoj se specificiraju interfejsi koje implementira a od kojih je zavisan. Predstavlja pogled na unutrasnjost komponente. Delovi klase nisu specifikacija instance. Do dela koji realizuje neki interfejs postoje kvadratici na ulazu (prikljuci). Prikljuci logicki grupisu interfejse.

52. Dijagrami rasporedjivanja *

Znacajni su za aplikacije koje se deplojuju na vise racunara. Vidi se fizicka organizacija naseg sistema sa aspekta broja racunara, komunikacionih puteva, nacina na koji su komunikacioni putevi realizovani. Sadrze uredjaje (cvorove) koje sistem sadrzi (serveri, radne stanice) i oni se posebno specificiraju na samom dijagramu. Ukoliko je bitno da su uredjaji redundantni specificira se i vrsta uredjaja. Sadrze artifakte (kakva konfiguracija na odredjenom racunaru mora da bude postavljenja, koji se podaci koriste itd...).

53. Dijagrami objekata/instanci *

Ima zadatak da prikaze presek objekata u nekom trenutku (instance). Moze se tretirati i kao dijagram komunikacija bez samih poruka a ima zadatak da se prakticno u njemu prikazuju instance na razlicite nacine (definisu se sa nazivom instance pa dvotacka ime klase, podvucene). Retko se koriste.

54. Dijagrami paketa *

Cesto se koriste u situacijama u kojima imamo slozene aplikacije. Paketi definisu neku grupaciju elemenata kako logickih tako i funkcionalno povezanih, predstavljaju neke

celine koje nam omogucuju da se spozna u kojim paketima imamo koje klase i na koji nacin paketi zavise jedno od drugih. Paketi predstalvjaju namespace-ove koji grupisu neke podpakete ili same klase. Na dijagramima se sam paket prikazuje na razlicite nacine, ono sto ih karakterise je drugi paket ili neki drugi sadrzaj (klasa u samom paketu). Zavisnosti izmedju paketa su predstavljeni strelicama sto znaci da paket iz koga izlazi strelica zavisi od paketa u koji ulazi strelica. Znacajni su za viseslojne aplikacije. Na osnovu ovih dijagrama mozemo utvrditi kakava strageija je dobra za dizajniranje softvera, gde obratiti posebnu paznju prilikom dizajniranja interfejsa. Paketi koji su najtrazeniji (koji su nasledjivani od drugih) veoma pazljivo treba da definisu interfejse jer ukoliko se tu napravi neka greska ona utice na veliki deo (ostatak) sistema.

55. Stilovi softverske arhitekture *

Mozemo reci da su stilovi vezani za to kako nas softver izgleda globalno, kakva je njegova struktura. Mogu se definisati kao paterni/obrasci. Predstavljaju “familiju sistema”. Odredjuju recnik i konektore koji se mogu koristiti uz neka ogranicenja. Uticu na kvalitet softvera koji razvijamo kao i brzinu kojom se softver razvija. Skup dizajnerskih odluka koje su kasnije skype za menjane! Kada se radi na vecim projektima koriscene stilova obezbedjuje zajednicki jezik, laksu komunikaciju i konverziju. Stilovi nisu iskljucivi moguce ih je kombinovati.

56. Podela stilova po kategorijama *

Stilove je moguce podeliti po razlicitim kategorijama.

- Komunikacionim procesima, SOA (servisno orijentisana arhitektura) i Arhitektura zasnovana na porukama (Message Bus)
- Rasporedjivanju, klijent/server, N-slojna i 3-slojna aplikacija
- Domenima, dizajn vodjen domenom (imamo velik broj aplikacija koje su danas toliko narasle u domenima kojima se bave da moraju specificno da se dizajniraju i specificiraju).
- Strutkturi, arhitektura zasnovana na komponentama, objektno-orijentisanim pristupima, slojevitoj arhitekturi i mikroservisna arhitektura.

57. Klijent/arhitektura *

Dvoslojna arhitektura, inicijalno je kreirana za aplikacije koje su imale korisnicki interfejs i bazu podataka. Moguce je imati vise klijenata i jedan server ili vise servera i jednog klijenta. Pod varijante ovog stila su:

- Klijent - Queue - Klijent sistem, svako od klijenata salje i prima poruke
- P2P (peer-to-peer), imamo komunikaciju izmedju razlicitih klijenata/aktera, sami vode racuna o svojim ulogoma. Decentralizovana arhitektura.
- Aplikativni serveri, centralizovana arhitektura, imamo server koji odradjuje neka procesiranja za koja su klijenti zainteresovani, web aplikacije su uglavnom bazirane na ovoj arhitekturi.

58. Prednosti i nedostaci klijent/server aplikacije *

- Visok nivo sigurnosti (svi podaci su na serveru gde je povecana sigurnost).

- Centralizovan pristup podacima (izmene i pristup podacima na serveru), jedan klijent ceka dok drugi menja itd…
- Lakse odrzavanje, jasno su podeljenje uloge i nadleznosti u okviru samog sistema. Analiza komunikacija u okviru ovakvog sistem se jako lako prati, propusti su jako lako uocljivi.
- Nedostatak moze biti bas to sto se podaci nalaze na jednom mestu, ukoliko dodje do ispada klijenti mogu da ostanu bez pristupa podacima. Zavisnost klijenata od centralnog servera je velika.

59. Arhitektura zasnovana na komponentama *

Komponente predstavljaju odredjene delove softvera koji su zaduceni za neke logicke operacije, imaju definisane interfejse i nacine funkcionisanja. Mogu se posmatrati i kao delovi softvera koji se mogu opciono plasirati na trzistu. U sustini je funkcionalna jedinica nekog softvera. Objektno orijentisana, medjutim predstavlja visi nivo apstrakcije jer definise odredjenu funkcionalnost. U distribuiranim sistemima komponente se mogu deploy-ovati na vise razlicitih mesta ali mogu se upotrebiti i na vise razlicitih okruzenja. Zamenjivost je veoma bitna i laka. Neka komponenta moze biti prosirenje nekoj drugoj komponenti.

60. Middleware *

Koristi se kao sloj izmedju komponenti, obezbedjuje njihovu komunikaciju

61. CORBA koncept *

Nastao je pre nekih dvadesetak godina. Omogucava da se izmedju klijenta i servera nalazi middlewate (ORB) koji zapravo ne dozvoljava da klijent i server ne komuniciraju direktno. Podaci se u klijentu preko object stub-a prosledjuju middleware-u dok se u serveru obradjuju preko object skeleton-a. Klijent i server nemaju direktan pristup jedan drugom, zavisnost od middleware-a je velika.

62. Prednosti i nedostaci koriscenja komponenti *

- Lako se rasporedjuju
- Manji su troskovi
- Visestruko koriscenje
- Otezano je odrzavanje jer nemamo punu kontrolu nad softverom, komunikacija se oslanja na middleware.
- Sigurnosni problemi su pristuni.

63. Dizajn vodjen domenom *

Predstavlja nacin na koji su organizovani podaci, biznis logika unutra same aplikacije. Podatke organizujemo tako da oni budu smernice za objektno orijentisani stil. Objekti unutar nase aplikacije treba da predstavljaju entitete u nasoj bazi podataka. Cesto ima kompleksan dizajn i takve aplikacije zahtevaju potpuno poznavanje sta u okviru naseg poslovnog domena treba da se izvrsava, koji elementi treba da postoje na nivou podataka i poslovne logike. Fleksibilan nacin organizacije i jednostavan za razvoj jer entiteti koji postoje u nasem sistemu se samo preslikavaju na neke realne objekte. Veoma je lako prosirenje, uvodjenjem novih entiteta ne remete se postojeci.

64. Slojevita arhitektura *

RAZVOJ VISESLOJNIH APLIKACIJA U EES

Predstavlja razdvajanje aplikacije na slojeve na osnovu funkcionalnosti, Slojevi mogu biti rasporedjeni na razlicitim ili istom racunaru. Ove aplikacije su veoma fleksibilne. Neke od prednosti/mana su:

- Imaju odredjenu apstrakciju, svaki sloj je izolovan.
- Ukoliko pricamo o distribuiranom sistemu izolovanost sloja se direktno povezuje sa skalabilnoscu, lako je dodavati nove slojeve.
- Visestruko koriscenje je moguce, pogotovo nad komponentama koje su blize klijentu.
- Definisu se standardi za komunikaciju.
- Nacin na koji se odrzava, ponekad odrzavanje oznacava kaskadne promene (vodopad), promena jedne strane moze da vodi do potrebe prosirivanja/izmene logike sa drudge strane.
- Nije adekvatna za jednostavne sisteme.
- Dodatni slojevi mogu narusiti performanse. Na primer dodavanje slojeva koji inicijalno nisu bili planirani, omogucavanje direktnom pristupu podacima itd..

65. Arhitektura zasnovana na porukama *

Bazirana je na tome da se u okviru sistema izmedju ucesnika u samoj aplikaciji razmenjuju neke poruke putem jednom/vise komunikacionih kanala. Obicno se koristi Publish/Subscribe (korisnici biraju za koje vrste podataka su zainteresovani, potencijalno mozda i od kog posiljaoca zele da dobijaju informacije, koliko cesto itd...) obrazac uz koriscenje Message Queuing-a. Arhitektura zasnovana na slojevima nam omogucuje:

- Komunikaciju zasnovanu na porukama
- Slozenu logiku procesiranja
- Modifikaciju logike procesiranja (od sinhronizma slanja i primanja poruka moze da zavisi da li je dobro istestiran neki vid komunikacije, jako je tesko otkriti neku bubu)
- Integracija sa razlicitim okruzenjima

66. Prednosti Message Bus stila *

- Jednostavnost, na vrhu je neki menadzer a ostatak sistema su radnici.
- Fleksibilnost, lako se dodaju novi tipovi poruka i/ili ucesnika.
- Velika skalabilnost
- Slaba povezanost, uglavnom se koristi neki message queueing.

67. Viseslojna (fizicka) arhitektura - N-tier, 3 -tier *

RAZVOJ VISESLOJNIH APLIKACIJA U EES

Slojevi predstavljaju delove aplikacije koji se izvrsavaju na odvojenim racunarima. Nesto sto je najcesca varijanta su 3 sloja, prezentacioni, biznis i sloj za pristup podacima. Da bi se ovakve aplikacije razvijale treba jasno definisati koje su funkcionalnosti neophodne, komponente koje bi funkcionalnosti podrzavale i nacin na koji bi se distribuirale. Osnovna logika je koliko slojeva treba da ima, sta ce biti na kojem sloju itd..Svaki sloj je nezavistan od drugih, osim sto komunicira, uglavnom asinrhonom sa susednim slojevima. Cesto primenjeno na bankarskim sistemima (web i mobile banke). Izuzetno je korisno sto jedan deo mozemo posebno da izolujemo (najcesce biznis, logicki deo). Neke od prednosti su:

- Lako odrzavanje.
- Skalabilnost.
- Fleksibilnost.
- Raspolozivost, dodavanje vise masina koje pokrivaju potencijalni pad, velika. Redudantnost.

68. Objektno orijentisani stil *

Funkcionalnosti se raspodeljuju na objekte (komponente su objekti). Objekti komuniciraju putem interfejsa. Objekti obavljaju odredjenje funkcionalnosti i zato su predstavljeni kao komponente. Objekti su prosirivi, moguce je visestruko koriscenje i jednostavni su za testiranje.

69. SOA (servisno orijentisana arhitektura) *

Funkcionalnosti se obezbedjuju skupom servisa, servisi koriste standardne interfejse. Karakteristike su:

- Autonomnost
- Distributivnost, postoje razliciti protokoli koji omogucavaju ovo primer je WCF
- Slaba povezanost, nezavisni jedni od drugih, treba da budu tako organizovani tako da su interfejsi kompatibilni sa okruzenjem.

70. Mikroservisna arhitektura *

Mikro servisi omogucuju da se nasa aplikacija izvrsava na okruzenju koje je u danasnje vreme definisano na razlicite nacine, poistovecuje se sa koriscenjem CLOUD-a. Service Fabrik arhitektura (ima u CLOUD skripti). Aplikaciju mozemo da razvijamo nezavisno od CLOUD-a a kasnije da je “pustimo” na CLOUD. Postoje dva tipa (Service Fabric):

- Reliable Actors, mogu biti statefull ili stateless, predstavljaju mikro servise koji imaju zadataka da odrade neku operaciju, jednostavnu operaciju.
- Reliable Services, mogu biti statefull ili stateless, imaju mogucnost da rade slozenije stvari.

71. Tehnike dizajniranja arhitekture *

Potrebno je jasno definisanje ulaza (slucajevi koriscenja, funkcionalni zahtevi, tehnoloski uslovi itd...). Sam proces dizajniranje je iterativni proces koji se koristi za formiranje arhitekture, jako je bitno jasno definisati ciljeve, vazne slucajeve upotrebe kao i kljucna pitanja.

RAZVOJ VISESLOJNIH APLIKACIJA U EES

72. Osnovne faze prilikom dizajniranja arhitekture softvera *

Postupak je iterativan zbog toga sto se najcesce desava da kroz sve korake treba proci vise puta kako bi dobili dobru arhitekturu sistema. Vrse se analize koje su horizontalne i vertikalne, horizontalne su po svakom sloju dok vertikalne podrazumevaju interakciju izmedju slojeva.

- Identifikovanje ciljeva, inicijalno jasno definisani ciljevi pomazu da se fokusiramo na resavanje pravih problema u nasem dizajnu. Precizni ciljevi znatno ubrzavaju proces. Takodje identifikuju se, u ovom koraku, i sami korisnici (arhitekte, testeri, menadzeri itd..), ogranicenja (softverska, hardverska ukoliko je hardver kupljen vec u napred i on nas uslovljava sa svojim specifikacijama).
- Definisanje kljucnih scenaraija (slucajeva upotrebe, skup scenarija koji definisu interakciju jednog ili vise ucesnika u sistemu), prakticno predstavljaju najznacajnija scnerarija za funkcionisanje softvera koji razvijamo. Usmeravaju arhitekturu u odredjenom smeru. Predstavljaju definiciju odredjenih problema i uticaje nekih karakteristika kao sto je bezbednost na sam sistem. Visok uticaj na kvalitet i funkcionalnost (Read, Update, Write).
- Kreiranje izgleda aplikacije, koji je tip aplikacije, koje tehnologije bi se koristile, koji je arhitekturalni tip. Identifikovati infrastrukturna ogranicenja. U ovom koraku se kreira neka slika/skica aplikacije koji bi predstavio opis komunikacije u samo aplikaciji.
- Identifikacija kljucnih pitanja, vezana su za neke stvari koje se ticu konkretno aplikacije, da li je ono sto smo unapred definisali zadovoljeno. Da li na primer moze doci do promena tehnologije, da li je potrebno kreirati aplikaciju tako da moze lako da se migrira izmedju tehnologija. Jako osetljiva stavka i kada se prica o arhitekturi ovde je veliki problem u razvoju aplikacije. Potrebno je definisati sta je to sto nasu aplikaciju cini dobrom, koji su to atributi koji uticu na kvalitet aplikacije.
- Definisanje kandidata/prototipa same arhitekture. Dobija se nakon definisanja kljucnih scenarija i kljucnih pitanja. Vrsi se validacija kandidata, sve stavke koje se proveravaju do koje mere su zadovoljene. Vrsi se revizija/iteracija jer je tesko zadovoljiti sve komponente.

73. Slojevite aplikacije *

Podela aplikacije po slojevima sa razlicitim funkcionalnostima. Predstavljaju logicku arhitetkruru sistema. Najcesca je troslojna aplikacija koja sadrzi sloj za prezentaciju, poslovni sloj (osnovna logika funkcionisanja sistema, sve za sta su klijenti zainteresovani se obradjuje na ovom nivou) i sloj podataka (omogucuje pristup nekim podacima, neka baza podataka ili server). Moze doci do pojavljivanja servisnog sloja, preko njega se definisu razni kanali za pristup biznis sloju, ponasa se kao dodatni posrednik (bolja sigurnost, klijenti koji se pojavljuju u spoljasnjem sistemu odvoje od logike, posrednik izmedju prezentacionog i biznis sloja).

74. Koraci projektovanja slojevite strukture *

- Izabrati strategiju slojevitosti (ovde se definisu nacini grupisanja funkcionalnosti po slojevima).
- Odrediti potrebne slojeve (kako i na koji nacin cemo odrediti potrebne slojeve).
- Kako distribuirati slojeve i komponente, distribuiranje slojeva treba raditi samo u situacijama kada je to neophodno, najcesce se radi kada zelimo da ispunimo neke zahteve bezbednosti (odvajanje prezentacionog sloja od logike npr....).
- Utvrditi da li treba spojiti slojeve, desava se u situacijama kada je logika prosta pa mozda taj sloj tu nije potreban.
- Odrediti pravila za intrerakciju izmedju slojeva, jednosmerna, dvosmerna itd…
- Indetifikovati poprecne nadleznosti, da li se neki delovi mogu ponovo koristiti.
- Definisati interfejse izmedju slojeva, jako je osetljiv, svaka izmena interfejsa je veoma osetljiva.
- Izabrati strategiju rasporedjivanja, na koliko mesta i gde staviti odredjene komponente.
- Izabrati komunikacione protokole izmedju slojeva, komponenti i slicno.

75. Alexander-ova misljenja
    - Dobre gradjevine imaju zajednicke osobine.
    - Procena da je gradjevina lepa nje samo stvar ukusa.
    - Traziti zajednicke osobine u okviru problema koji se resavaju.

76. Alexander-ova definicija obrasca *

Svaki obrazac opisuje problem koji se stalno pojavljuje u nasem okruzenju i zatim opisuje srz resenja tog problema tako da to resenje mozemo koristiti million puta na razlicite nacine.

77. Razlozi za koriscenje softverskih obrazaca *

Najcesce se ne isplati trosenje vremena na resavanje nekih problema i zbog toga implementiramo neke obrasce koji predstavljaju neka dobra resenja slicnih problema. Uspostavlja se komunikacija, koristi se neka opsta terminologija, neko drugi kada pogleda nase resenje ce se mnogo lakse snaci.
Posmatranje problema sa viseg nivoa, projektovanje. Kada koristimo neke predefinisane obrasce dobijamo osobinu povecane fleksibilnosti.

78. Objekti kroz konceptualni pristup
    - Tradicionalni pristup, objekti su podaci sa metodama.
    - Konceptualni pristup, objekti su entiteti sa odgovornostima, odgovornosti odredjuju ponasanje objekta.

79. Kapsuliranje

Bazira se na tome da koristimo neke apstraktrne klase. Omogucava lakse odrzavanje softvera. Skrivenu implementaciju mozemo lako da menjamo. Interfejsi diktiraju koje su to odgovornosti koje nase klase treba da ispunjavaju.

80. Gde projektovati kapsuliranje

U fazi dizajna potrebno je razmisljati sta je potrebno da se menja u projetku. Potrebno je razdvojiti promenljive delove i posmatrati ih odvojeno u smislu da ih na neki nacin kapsuliramo.

81. Projektovanje pomocu obrazaca *

RAZVOJ VISESLOJNIH APLIKACIJA U EES

Potrebno je razumeti konstekst problema i iskoristiti obrasce za njegovo resenje, projektovanje je cesto proces sinteze, spanja i kombinovanja elemenata.
Nezavisno konstruisanje delova i njihovno naknadno sklapanje nike elegantno resenje.
Kvalitetan projekat predstavlja presek nekih obrazaca i onda te obrasce dodajemo u pravilnom redosledu.
Osnovni pristup je razdvajanje kompleksnih delova na jednostavnije koje se potom medjusobno dalje dopunjuju i razvijaju.

82. Koraci pri pojektovanju *

- Konceptualno definisati problem i cilj projekta.
- Odrediti obrasce (obrasci koji predstavljaju odnos izmedju elemenata/entiteta).
- Poceti od obrazaca koji definisu kontekst za ostale obrasce treba prvobitno da budu definisani.
- Postupak definisanja i implementiranja obrazaca se ponavlja.
- Usavrsavanje i realizovanje projekta.

83. Karakteristike softverskih obrazaca *

- Ime, jedinstveni identifikator samog obrasca. Ime je cesto definisano tako da dodatno upucuje na to koja je namena samog obrasca (koji problem on resava).
- Namena, odnosi se na svrhu obrasca, u kom kontekstu se koristi.
- Problem, definicija problema koji obrazac pokusava da resi.
- Resenje, nacin na koji je obrazac resio problem.
- Ucesnici i saradnici, entiteti koji se koriste unutar obrasca.
- Posledice, uticaji koji deluju na sam obrazac.
- Realizacija, kako se i na koji nacin sam brazac realizuje.

84. Klasifikacija obrasca *

- Na osnovu domena (nivoa apstrakcije), mogu biti obrasci kod kojih koristimo klase ili objekte.
- Na osnovu namene, tu obrasci mogu da definisu neko kreiranje objekta (creational patterns), obrasci koji definisu strukturu (klase), nazivaju se structural patterns i obrasci koji definisu ponasanje unutar klasa (template patterns)

85. Unikat (Singleton, kreatorski obrazac) *

Zamislimo da imamo jedan serverski objekat i vise klijentskih objekata koji mu pristupaju.
Imajuci u vidu da klijenti dele informacije medju sobom (neko salje a neko cita sa servera) i svi oni pristupaju samo jednom serveru. Pitanje je kako sinhronizovati komunikaciju izmedju servera i klijenata prilikom startovanja servera.
Singleton nam govori bas to, na koji nacin i sta je potrebno implementirati na serveru kako bi se obezbedila funkcionalnost jedne instance klase sa globalnim pristupom.
Klijent dobija pravu instancu tipa Singleton preko metode GetInstance() bez da vodi racuna da li ona postoji ili ne.

86. Realizacija Singleton-a *

Kreiramo metodu koja proverava da li je instanca formirana ili je NULL. Ukolko je NULL kreiramo instancu. Problem nastaje kada imamo vise klijenata, ukoliko se pozove metoda od strane vise klijenata u istom trenutku svakom od njih se vraca nova instanca sto nije ono sto zelimo, mi zelimo da imamo samo jednu instancu za svakog od klijenata. Jedno resenje moze biti imelementiranje reda pozivanja metode sto je dobro za prvi poziv ali kasnije on postaje bespotreban. Optimalno resenje bi bilo da koristimo regione (atomski regioni) i mogucnost njihovog zakljucavanja. Primenjuje se dublo zakljucavanje, ukoliko oba klijenta prodju prvi uslov i dodju do drugog dela provere samo jedan klijent moze da kreira instancu dok se drugom samo prosledi.

87. Komanda (Command) *

Drugo ime je akcija/transakcija. Namena ovog obrasca je da vraca stanje koje je postojala pre izvrsavanja zahteva preko undo operacije. Kada imamo multikorisnicki rad ili mogucnost da se vise komandi pojavljuju od strane jednog klijenta njihovi zahtevi se pretvaraju u objekte koji su organizovani tako da predstavljaju nasledjenu klasu klase komanda sa operacija execute() i undo().

88. Adapter *

Predstavlja nesto sto se koristi za prilagodjavanje raznih aspekata u nasem zivotu (punjenje telefona, slusalice itd...). Konkretno adapter koristimo da neki interface pretvorimo u interface koji klijent ocekuje. Moze se tretirati kao arhitekturalni obrazac, mogu da postoje citavi sistemi koje je potrebno da se sjedine. Nije potrebno menjati klasu naseg sistema kao ni klasu sistema sa kojim zelimo da izvrsimo “spajanje”, dodatni kod i sve promene se vrse u adapter-u.

89. Sablonski metod (Template method) *

Ukoliko postoje “dva slicna recepta, primer kafa i caj” (klase koje koriste slicne metode) potrebno je keirati opstiju sablonsku, apstraktnu klasu koje obe klase (recepti) mogu/moraju da implementiraju naravno uz neke svoje dodatne/specificne metode. Kod ovog obrasca cesto su prisutne “hook” operacije, najcesce su prazne ali mogu da definisu neko standardno ponasanje. Opcione operacije koje mogu ali ne moraju da se definisu za razliku od apstraktnih. Najcesce ima primenu u klasama koje ne zele da redefinisu operacije pa iskoriste prazan “hook”.

90. Posmatrac (Observer) *

Definise zavisnost objekata (jedan sa vise ili vise sa vise). Po promeni stanja jednog objekta obavestavaju se zavisni objekti. Naziva se i Publish-Subscribe obrazac. Moze se dogoditi da vise posmatraca slusa jednog klijenta koji azurira neke podatke (prikazuju iste podatke na razlicite nacine), priikom promene nekog podatka svi posmatraci moraju biti obavesteni kako bi ih korektno prikazali. Struktura obrasca je sacinjena od subjekta koji predstavlja interfejs za povezivanje i raskidanje veze izmedju konkretnog subjekta i posmatraca. Observer/Posmatrac koji sluzi da izvrsi azuriranje posle neke promene. Konkretni posmatraci odrzavaju referencu na konkretni subjekat i cuvaju stanje za koje su zainteresovani i implementiraju interfejs posmatraca za azuzriranje. Promena ne mora svaki put da bude poslata posmatracima, zavisi naravno od konfiguracije sistema, najcesce ukoliko je promena mala ona se ne salje odmah.

91. Apstraktna fabrika *

Interfejs za pravljenje familija povezanih objekata (klijenti ih nisu svesni). Primer je softver za auto servise. Izoluje konkretne klase i olaksava zamenjivanje familije proizvoda. Unapredjuje se konzistenstnost proizvoda i jasno definisana procedura za njegovo pravljenje. Mana ovog paterna je to sto postoji relativno velik broj klasa i svaki novi proizvod bi zahtevao promenu interfejsa fabrike sto dovodi do kaskadnih promena i otezava odrzavanje.

92. Stanje *

Omogucava da objekat promeni ponasanje kada mu se promeni unutrasnje stanje. Primer moze biti mrezna konekcija sa vise stanja povezanosti. Omogucen je eksplicitni prelazak iz jednog stanja u drugo stanje. Kada u aplikaciji imamo potrebu da imamo prelaske nekih stanja onda se oni definisu na nacin da se u konfiguraciji (nekom konfiguracionom fajlu) definisu mogucnosti prelazaka iz jednog stanja u drugo. Struktura obrasca je jednostavan, imamo context mesto gde cuvamo stanje, context se referencira na stanje (apstraktnu klasu). Konkretna stanja su vezanja na apstraktnu klasu stanja. Ovaj obrazac lokalizuje ponsanje nekog objekta na taj nacin sto se u okviru jedne klase implementiraju sva stanja (izbegavaju se glomazni uslovi).

93. Strategija *

Koristi se u situacijama kada zelimo da definisemo neku familiju algoritama. Omogucava da se jednostavno primene algoritmi u toku izvrsavanja softvera. Svaki od algoritama se enkapsulira i moze da se primeni u svakom trenutku u zavistnosti od potreba. Problem je prosirivost i ona se resava uvedjenjem “specificnijih” klasa ponasanja (primer patkice). Dobra praksa je razdvajanje promenjivih od nepromenjivih delova radi lakseg kasnijeg odrzavanja i prosirivanja koda. Klijent mora biti svestan razlicitih strategija, dodatno opterecenje komunikacijom izmedju strategije i konteksta i uvecan broj objekata mogu biti neke od mana ovog paterna.

94. Muva *

Koristi se u situacijama kada imamo potrebu za rukovanjem velikim brojem objekata. Ne pravi pojedinacne instance za svaki od potrebnih objekata (primer je editor teksta). Njena glavna karakteristika je deljeni objekat. Flyweight pool predstavlja bazen state-free (bez specificnih vrednosti atributa) objekata koji se kasnije po potrebi popunjavaju i koriste (referenciraju). Glavna prednost ovog obrasca je smanjenje memorije, manje objekata se kreira i koristit ali i objedinjuju se stanja veceg broja objekata. Mane su nemogucnost nezavisnog upravljanja instanci u okviru muve.

95. Dekorater *

Strukturni obrazac kada zelimo dinamicki da pridruzimo neke dodatne funkcionalnosti/odgovornosti objektu. Koristi se kod multi-layer struktura (na primer korisnicki interfejs gde imamo razlicite karakteristike koje su deo jedinstvenog pogleda korisnika na samu aplikaciju). Treba ga organizovati tako da dizajn koji podrazumeva sadrzavanje korisnickog interfejsa bude lako prosiriv i da bude lako adaptivan (kada na primer koristimo word imamo mogucnost ukljucivanja lenjir trake ili ne). Svaki od dodatnih odgovornosti pakujemo u jedan objekat koji potom dinamicki mozemo da dodajemo nasoj aplikaciji ukoliko je to potrebno. Objekti se hijerarhijski referenciraju.

- Open-Closed princip, baziran na tome da klasa treba da bude otvorena za prosirenje a zatvorena za modifikaciju.

Mana na primer predstavlja to sto mozemo da imamo puno malih odgovornosti/malih objekata kao i da dekorater i komponenta nisu istog tipa (nisu identicni) pa polimorfizam nije adekvatno moguce iskoristiti.

96. Proizvodni metod (Factory Method) *

Radi se o metodi za pravljenje objekata, odnosno definisanje interfejsa u okviru kojeg se nalazi metoda za pravljenje objekta. Cilj ovog obrasca je da se odvoji objekat koji se pravi od klijenta odnosno implementacija koju klijent poziva od njega samom. Na taj nacin se vrsi enkapsulacija implementacije podataka. Nedostatak je to sto moze da postoji relativno velik broj klasa koji se implementira jer je potrebno vise kreatora, jedan kreator za jedan objekat.

97. Proxy obrazac (posrednik) *

Strukturni obrazac koji ima zadatak da posreduje izmedju nekog objekta na serveru i objekta koji koristi taj server/servis u cilju implementacije neke dodatne kontrole pristupa ili mogucnosti analize klijenata koji pristupaju serveru ili neke dodatne evidencije koliko se klijenata prijavilo na sam servis itd...Vrste proksija:

- Daljinski (Remote Proxy), omogucava udaljen pristup nekom objektu najcesce se koristi kod distribuiranih aplikacija. Pristup udaljenom objektu koriscenjem proksija se posmatra kao lokalna operacija (imamo osecaj da se server nalazi na nasoj masini jer se proksi nalazi na njoj).
- Virtuelni/lenji (Virtual Proxy), odlaze neke skupe inicijalizacije objekata. Kada hocemo da pristupimo/ocitamo neku veliku sliku onda se u okviru takve operacije moguce cuvati neke informacije vezane za sliku cime se omogucava pametno preuzimanje svih podataka/informisati klijenta da su svi podaci ucitani i ako u stvarnosti nisu.
- Zastitni (Protective Proxy), omogucava proveru prava pristupa nekom objektu. Posmatra se kao barijera nekog objekta, nije dozvoljeno makar kakvom klijentu da pristupi svakom objektu.
- Pametni (Smart Proxy), omogucava dodatnu logiku prilikom pristupa klijenata serverskom objektu, uglavnom se koristi za sinhronizaciju pristupa i brojanje referenci. Omogucava prikupljanje nekih dodatnih informacija koje mogu biti vazne za sistem.

Neki stvarni objekat definise akcije koje zeli da se izvrse, one prolaze preko proksija koji ima neki preprocessing/dodatnu logiku.

98. Iterator *

Obrazac koji omogucava laksi pristup nekim kolekcijama podataka, sekvencijalno pristupanje elementima iz tih kolekcija, eventualno paralelno (vise istovremenih obilazaka) sekvencialno pristupanje.

99. Prototip *

Kreatorski obrazac koji omogucava da se kada u situacijama u kojima imamo dosta objekata koji lice jedan na drugi (isti su) novi objekti prave kopiranjem objekta prototipa. Koristi se postojeci prototip (objekat) i novi se formiraju na osnovu njega (primer note i notne linije).

- MemberwiseClone(), plitko kopiranje (shallow copy) ne kopiraju se referencirani objekti.
- IClonable, implementira nacin kako ce se praviti nova instanca od postojece i omogucava da se podesi da li ce klon biti kao deep ili shallow copy.

100. Sastav (Composite) *

Ovaj strukturni obrazac ima zadatak da sastavi objekte u hijerahisku strukturu koja bi mogla da se prikaze kao neki tip stabla. Ono sto je karakteristicno za ovaj obrazac je da klijenti mogu uniformno da tretiraju i proste i slozene objekte u okviru ove strukture (primer direktorijuma i editora za crtanje). Obrazac omogucava dodavanje potomaka i njihovo brisanje. Mana moze biti da moze da dovede do previse uopsten dizajna gde se neka ogranicenja ne mogu slobodno postaviti (ne mogu se ukljuciti samo odredjenje komponente).

101. Spajanje paterna

Podrazumeva kombinaciju paterna i njihovo medjusobno ukljucivanje u resenje cesto dva ili vise. Postoje grupe paterna koje najcesce prate jedni druge (Komanda i Observer se cesto koriste zajedno).

102. Slojevite aplikacije

Predstavlja podelu sistema po slojevima, izbor slojeva podrazumeva da imamo sliku o tome koji slojevi mogu biti prisutni u aplikacijama i na osnovu toga definisemo dizajn same aplikacije. Predstavlja najvisi nivo apstrakcije, logicku strukturu sistema. Zadatak prilikom razvijanja slojeva je njihovo prepoznavanje i grupisanje. Najcesce se koristi troslojna aplikacija gde se podrazumeva koriscenje:

- Prezentacionog sloja, pruzamo neke usluge, omogucavamo pristup servisima.
- Poslovni/Business sloj (domenski sloj), sloj koji sadrzi logiku funkcionisanja sistema, sta i kako nasa aplikacija treba da radi.
- Sloj podataka, predstavlja mogucnost komunikacije sa nekim izvorom podataka najcesce bazom.

Nesto sto se cesto pojavljuje kod viseslojnih aplikacija je neki SERVISNI sloj koji omogucava pristup nekim eksternim sistemima, omogucava neke dodatne kanale za pristup poslovnom sloju.

103. Tipovi klijenata

- Debeli, karakteristicni za situacije kada zelimo da imamo neku logiku proracuna, obrade podataka ili validacije koja se izvrsava na samom klijentu. Jedan od razloga moze biti sto zelimo da imamo sto brzi odziv na neke promene u sistemu (SCADA sistemi).
- Tanki (mobilni uredjaji), nemaju mogucnost izvrsavanja nikakve logike, jednostavan hardver, ne cuvaju (kesiraju) podatke.

104. Koraci projektovanja slojevite strukture
    - Izabrati strategiju slojevitosti, postoje razliciti nacini grupisanja, broj racunara koji mozemo koristidi itd...
    - Odrediti potrebne slojeve, kako grupisati funkcionalnosti po odredjenim slojevima, da li poslovni treba biti odvojen od prezentacionog itd…
    - Odluciti kako distiribuirati slojeve i komponente, da li cemo sistem moci da prosirujemo sa vise racunara (imacemo redundantne/replikacione slojeve, skalabilnost)
    - Da li treba spojiti slojeve, odnosi se na sam tip aplikacije i uslove pod kojima treba definisati da li je poslova logika i poslovna pravila takva da predstavljaju neke jednostavne akcije (ako su u pitanju samo validacije i slicno onda mozda i nije neophodno da postoji poslovni sloj).
    - Odrediti pravila interakcije izmedju slojeva, mogu da budu razlicita, najcesce su top-down, moze da bude i jednosmerna (komunicira se samo sa slojem ispod) ali moze da bude i takva da slojevi komuniciraju i sa slojem ispod i iznad.
    - Identifikovati poprecne nadleznosti, aspekti viseslojnih aplikacija koji su vezani za potrebe svakog od sloja (bezbednost itd...).
    - Definisati interfejse izmedju slojeva (WCF na primer).
    - Izabrati strategiju rasporedjivanja, preporucuje se da se koriste neki paterni za deployment koji su opste prihvaceni.
    - Izabrati komunikacione protkole, biraju se na osnovu vise kriterijuma, na primer programskim jezikom, nasim sklonostima, fizickim ogranicenjima i slicno.

105. Dizajn prezentacionog sloja

Moze se podeliti na komponente koje se vezuju za korisnicki interfejs (UI) i komponente prezentacione logike (obrasci kao sto je MVC na primer).

106. Uticaj na projektovanje prezentacionog sloja

- Kesiranje, radi se zbog povecanja performansi (MVC obrazac). Brzi odziv i pristup podacima od interesa. Ukoliko su podaci kriticni sa aspekta bezbednosti i ako govorimo o visekorisnickom pristupu onda korisnici sa drugim korisnickim pravima (ukoliko se podaci kesiraju) mogu da im pristupe sto je lose, u tom slucaju nije dobro koristiti kesiranje.
- Komunikacija, odnosi se na asinhrone operacije, komponente u okviru sloja ce ocekivati neku notifikaciju (callback) itd...Uglavnom se koristi neki multithread-ing sistem, dobija na znacaju kada se prikupljaju velike kolicine podataka (paketi se dele na manje kako ne bi doslo do gusenja mreze, podaci pristizu asinhrono).
- Kompozicija, treba izbegavati veliku zavisnosti izmedju komponenata, ukoliko treba organizovati neki nacin komunikacije sa okruzenjem koristi se standardizovan pristup (pub-sub).
- Rukovanje izuzecima, kako ce se izuzeci propagirati i na koji nacin ce mu biti prikazivani.
- Navigacija
- Korisnicko iskustvo, da li korisnik ima iskustva u radu sa slicnim sistemom/aplikacijom itd…
- Korisnicki interfejs, uslovljen zahtevima klijenata koji narucuju aplikaciju ili deo potrebe za pravilnim funkcionisanjem programa.
- Validacija

107. Koraci u dizajniranju prezentacionog sloja
    - Identifikovati tip klijenta
    - Izabrati tehnologjiu
    - Dizajnirati UI
    - Odrediti strategiju validacije podataka
    - Odrediti poslovnu logiku, da li ce se poslovna logika ugraditi u prezentacioni sloj ili ce biti na drugom racunaru.
    - Odrediti komunikacionu strategiju
108. Projektantska pravila za poslovni sloj
    - Da li je potrebno izdovjiti poslovni sloj
    - Ukoliko je neophodan potrebno je definisati nadleznosti i korisnike poslovnog sloja, najteze je prepoznati sta je poslovna logika za odredjenu aplikaciju
    - Ne mesati razlicite vrste komponenti za resavanje problema
    - Smanjiti komunikaciju ka udaljenom poslovnom sloju ukoliko se radi o fizicki odvojenim masinama.
    - Izbegavati cvrstu povezanosti izmedju slojeva, slojevi ne bi trebalo direktno da zavise jedno od drugog, da na primer komponente nekog sloja rade na osnovu podataka koje dopremljuje neki drugi sloj.

109. Posebni projektantski zahtevi

- Autentifikacija, da li je klijent bas taj klijent (najcesce neko logovanje).
- Autorizacija, koja ovlascenja/nadleznosti ima korisnik.
- Kesiranje, podrazumeva da imamo neki kes u okviru kojeg se podaci kontinualno/asnhrono prikupljaju i onda se takvi podaci prezentuju klijentima na njihov zahtev. Kes takodje omogucava klijentima da menjaju podatke (sinhronizovanim/zakljucanim pristupom) sto direktno uzrokuje izmene podataka na bazi podataka.
- Povezanost (coupling) i kohezija (lepljenje), ukoliko definiseno neku komponentu sa odredjenim atributima onda se u okviru metoda koje klasa sa tim atributima implementira treba uvek koristiti koheziju (bas te atribute koji su definisani, podaci se koriste na mestu gde se i nalaze). Coupling je vezan za komunikaciju izmedju raznih komponata/modula, ukoliko se nalaze na razlicitim racunarima/lokacijama onda on treba da bude sto manji kako bi zavisnost izmedju modula i samih slojeva bio sto manji.
- Rukovanje izuzecima
- Logovanje, ukoliko klijent koji koristi neku nasu aplikaciju i utvrdi da je neki nas servis pao, preko logovanja podataka o prekidu se moze utvrditi tacno mesto i urzrok zbog cega se dogodio problem. Omogucava mogucnost lake replikacije kako bi lakse popravili sistem. Moze uticati na performanse i zbog toga se loguju samo problematicni delovi koda, najcesce oni koji su vezani za izuzetke.

110. Trace level
    - Logovanje kada se dogodi neka greska - ERROR.
    - Logovanje kada se desi neko upozorenje - WARNING.
    - Kada zelimo da proverimo kako aplikacija radi u nekom specificnom slucaju - DEBUG.
    - Logovanje informacija o radu sistema - INFO

    Nivo trace-a se konfigurabilno postavlja u zavisnosti od nase aplikacije. Uglavnom se koriste nivoi koji odredjuju sta se prati i loguje, na primer nivo dva bi logovao samo ERROR-e i WARNING-e, inicijalno to je uvek bas i nivo dva.

111. Koraci u dizajniranju poslovnog sloja
    - Kreiranje dizajna visokog nivoa
    - Projektovanje poslovnih komponenti
    - Projektovanje poslovnih entiteta
    - Projektovanje toka poslovnog procesa (workflow)

112. Koraci u projektovanju poslovnih komponenti

- Identifikacija komponenata, identifikacija poslovne logike (pravila ponasanja, ponasanje aplikacije, odrzavanje konzistentnosti)
- Poslovni entiteti, modelovanje domena za poslovnu logiku i stanja
- Poslovni tokovi (workflow), da li su to visestepeni procesi, da li se zahteva neka komunikacija izmedju vise komponenti itd…
- Donosenje kljucnih odluka za poslovne komponente, na koji nacin se razmenjuju poruke, da li je event base komunikacija, da li su klijenti debeli ili tanki itd…
- Izbor vrsta transakcije
- Identifikovati upravljanje poslovnim pravilima, da li su komponente poslovne logike neke web aplikacije ili servisi, ko ih pokrece i na koji nacin se izvrsavaju
- Identifikovanje potrebnih softverskih obrazaca (Komanda, Fasada, Adapter, Factory)

113. Projektovanje entiteta
    - Cuvaju vrednosti podatka
    - Validacija podataka
    - Izbor formata, korisnicki objekti, XML, DataSets
    - Izbor tipa entiteta
114. Dizajniranje toka poslovnih procesa
    - Sekvencijalni (Sequential Workflow), unapred zadat niz aktivnosti, zadatak se pomera kroz odredjeni skup koraka dok se ne izvrsi.
    - Stanjem vodjen (State-Driven Workflow), u zavisnosti od nekog dogadjaja prelazi se u drugo stanje
    - Podacima vodjen (Data-Driven Workflow), oredjen nekim vrednostima u sistemu (visoke, jako niske itd...)
    - Manuelnim vodjenjem (Human Workflow)
115. Transaction Script

    Proceduralni stil (vise koraka, vise komponenti i resursa za ivrsavanje nekog zadatka) - pre nego OOP, predstavlja jednu proceduru koja se odnosu na jednu/svaku poslovnu transakciju. Svaka procedura sadrzi svu biznis logiku potrebnu za obavljanje kompletne transakcije (workflow, pravila, validacija, perzistencija u DB). Grupise se u neku vrstu statickog menadzera ili servisa. Prednosti su jednostavnost modela, ocigledno je gde se postavljaju granice za otvaranje i zatvaranje transakcije. Nedostaci mogu biti povecanje slozenosti logike domena i potencijalno dupliran kod (na vise komponenti koje reade slicne stvari ili u okviru jedne komponente) koji je tesko uocljiv.
116. Domain Model Pattern

    Poenta je izgraditi objektni model u okviru domena koji sadrzi podatke ponasanja. Modeli ponasanja i podataka su asocirani sa nekim realnim objektima iz klijentskog okruzenja. Omogucavaju nam da u poslovnom sloju imamo podatke koji mogu da simuliraju podatke koje se nalaze u bazi podataka, poslovni sloj izgleda kao poslovni sloj sa podacima koji se koriste u realnom svetu. Polazi se od pretpostavke da ne postoji baza podataka.

117. Table Module

- Mapira objektni model na model baze, jedan objekat predstavlja tabelu u bazi
- Jedan objekat predstavlja tabelu u bazi
- Objekat je odgovoran za sve potrebne perzistencije
- Ako se objektni model i model baze pocnu razdvajati vreme je za refaktorizaciju

118. Dizajn sloja podataka

Predstavlja sloj kod koga imamo mogucnost pristupa nekim izvorima podataka na osnovu poznavanja njihove destinacije i opisa samih podataka u njima. Najcesce komponente koje se ovde pojavljuju omogucuju i sam pristup tim podacima. Pristup je centralizovan, omogucena je sinhronizacija i konzistentnost podataka. U okviru ovih komponenti je omogucena i logika pristupa podacima u cilju efikasnijeg pristupa podacima.

119. Servisni agenti

Upravljaju komunikacijama sa odredjenim servisima iz susednih slojeva, u okviru ovih servisa se obezbedjuju razliiti zahtevi kao sto su mapiranje podataka, prilagodjavanje, kesiranje u cilju efikasnijeg preuzimanja itd...U sistemima u kojima postoji velika kolicina podataka i veliki broj klijenata koji im pristupaju i menjaju kesiranje je neophodno.

120. Projektantske odluke u sloju podataka

- Izbor tehnologije (ADO.NET npr.)
- Interfejsi, potrebno je izabrati odgovarajuci vid apstrakcije za pristup sloju podataka. Kako ce se oni organizovati, kome su namenjeni, da li ce ih biti vise itd…
- Enkapsulacija, pozeljno je implementirati nad nekim funkcionalnostima za pristup bazama.
- Mapiranje podataka, treba odluciti kako mapirati entitete aplikacije.
- Azuriranje podataka, kako i na koji nacin se azuriraju, da li se grupisu. (BATCHING) ili se azuriraju pomocu nekih transakcija ili store procedura itd…
- Upravljanje konekcijama, kako ce se upravljati konekcijama, da li se konekcija otvara i zatvara prilikom svakog pristupa ili ce postojati neka logika radi optimizacije sa radom sa bazom.
- Rukovanje izuzecima, da li ih treba koristiti i na koji nacin.
- Bezbednost

121. Dizajn servisnog sloja

Ima zadatak da bude medijator/posrednik izmedju prezentacionog i poslovnog sloja i da po potrebni omoguc nekim spoljasnim sistemima/servisima pristup nasoj aplikaciji u cilju obavljanja odredjenih zadataka. Obicno sadrzi:

- Interfejse servisa.
- Tipove poruka, strukture sa podacima iz poruka da bi se komunikacija mogla izvrsiti.

122. Projektantska razmatranja u servisnom sloju

- Projektovanje servisa sa aspekta aplikacije a ne komponenti, znacajno je u cilju razvijanja efikasnog okruzenja. Sa aspekta viseslojnih aplikacija imaju zadatak da vise paznje posvete nekom posredovanju u komunikaciji i potrebi ostalih komponenti.
- Dizajnirati prosirive servise bez razmatranja ko su klijenti.
- Koristiti standardne elemente, za servise se vrlo cesto koriste standardne specifikacije koje propisuju nacin pristupanja odredjenim aplikacijama. Omogucava klijentima ukoliko jednostavno isprate standard lak pristup aplikaciji.
- Projektovati sa pretpostavkom dobijanja nevalidnih zahteva, ukoliko klijent posalje neki zahtev koji nije validan oni mogu biti izuzetno opasni po sistem i jako je vazno da se pravilno procesiraju. Ovaj sloj je veoma osetljiv sa aspekta bezbednosti.
- Mogucnost da se detektuje da poruka nije poslata i da se posalje ponovo (idempotencija). Podrazumeva da imamo neku dodatnu funkcionalnost koja bi omogucila da se modeluje taj neki PUB-SUB mehanizam za razmenu poruka.
- Mogucnost upravljanja redosledom poruka (komutativnost), najcesce se koriste redovi (red cekanja sa prioritetima) za izvrsavanje ovakih zahteva, ukoliko postoje neki klijenti veceg prioriteta (admini) potrebno je omoguciti visi prioritet od klijenata koji su vezani za neke spoljasnje aplikacije/klijente.

123. Projektantski zahtevi servisnog sloja
    - Autentifikacija
    - Autorizacija
    - Komunikacija
    - Rukovanje izuzecima
    - Kanali poruka
    - Konstruisanje poruka
    - Zastita poruke
    - Rutiranje poruke
    - Interfejs servisa
    - Validacija

124. Rasporedjivanje slojeva
    - Nedistribuirana arhitektura, svi slojevi su na istoj masini (eventualno izbor podataka moze biti na nekoj drugoj). Jednostavnog i performanse izmedju slojeva su prednost. Mana je to sto svi slojevi dele resurse sa cime je njihova opterecenost veca a efikasnost potencijalno manja. Ako se masina pokvari ceo sistem prestaje sa radom.
    - Distribuirana arhitektura, vise fizickih masina na kojima se nalaze slojevi. Imamo mogucnost obezbedjivanja vece skalabilnosti i bezbednosti u situacijama kada slojevi koji su vezani za poslovnu logiku treba da realizuju vecu bezbednost od prezentacionih. Narucilac aplikacije moze da diktira ono sto zeli a da se neko ko aplikaciju razvija mora prilagodjavati zahtevima narucioca predstavlja jednu od prednosti

125. Preporuke za lociranje komponenta

- Distribuirati komponente sam ako je neophodna veca bezbednost i skalabilnost.
- Sinhrono koriscenje prezentacionih i poslovnih komponenata (bolje performanse na istoj masini zbog cestih komunikacija izmedju slojeva)
- Razliciti bezbednosti nivoi - razlicite masine
- Postaviti poslovne komponente koje se pozivaju asinhrono, zajedno sa workflow komponentama u poseban sloj.
- Postaviti poslovne entitete na isti fizicki sloj kao i komponente koje ih koriste.

126. Obrasci performansi - Load Balancing

Ukoliko sami razvijamo LOAD BALANCING moramo da vodimo racuna o tome kako te masine/aplikacije treba da izgledaju. Ponekad se moze primenjivati kada delimo neke poslovne zadatka (na primer vise servera koji ravnopravno izvrsavaju neke zadatke koje im prosledjuje neki manager u kojem se nalazi neka logika balansiranja). Ukoliko govorimo o pracenju sistema ili nekih delova mreze (EES) onda bi BALANCER mogao da se izvrsava tako sto bi svaki server dobijao odredjenje zadatke u okviru mreze (sortiranje po kompleksnosti).

127. Obrasci raspolozivosti - Failover Cluster

Ukoliko dodje do otkaza nekog servera koji klijenti koriste moguce je preusmeriti njihovu konekciju na “vruce” rezerve/replike kako im seris ne bi bio ukinut. Danas visoko zahtevano u svim ozbiljnijim distribuiranim aplikacijama.

128. Projektovanje kompnenata

Postoji veliki broj komponenata u okviru svakog sloja viseslojnih aplikacija, izuzetno je bitno pravilno ih projektovati. Bilo da se radi o prezentacionom sloju ili servisnom, u okviru svakom od njih je potrebno voditi racuna kako sastavne komponente treba da budu dizajnirae i kako ih projektovati da budu dobro dizajnirane.

Cross-Cutting komponente su zaduzene za bezbednu komunikaciju i neke zadatke operacionog menadzmenta kao sto su logovanje, performance counting i slicno.

129. Karakteristike lose dizajniranog softera *

- Krutost (Rigidity) koda, problem sa prosirivoscu. Dizajn je tesko promeniti a kod jako tesko prosiriti, bilo koje dodavanje nove funkcionalnosti je jako tesko izvodljivo.
- Krhost (Fragility), podloznost gresaka. Dizajn je jako lako “razbit”, promena u jednom delu koda dovodi do promene u nekom trecem (neocekivane promene).Kod “puca” na neocekivanim mestima.
- Nemobilnost (Immobility), nemogucnost prenosenja u neko drugo okruzenje. Dizajn je zamrsen i t esko ga je rastaviti na prenosne delove.
- Viskoznost (Viscosity), velika povezanost izmedju komponenata i dugo kompajliranje. Viskoznost softvera podrazumeva da u okviru softvera imamo veoma zavisne objekte (jedan koristi podatke drugog i niko drugi). Viskoznost okruzenja predstavlja dugo kompajliranje i dugotrajno testiranje (integracija vise projektnih timova).
- Nepotrebna slozenost, u situacijama kada se dodaju neki delovi koji ce “mozda trebati” i tesko razumevanje koda (povecana necitljivost i otezano odrzavanje).
- Nejasnoca (Opacity), ne ispravljanje svojih i tudjih gresaka.

130. Smernice prilikom dizajniranja komponenti

RAZVOJ VISESLOJNIH APLIKACIJA U EES

Primeniti SOLID principe:

- S - Single Responsibility Principle
- O - Open Closed Principle
- L - Liskov Substitution Principle
- I - Interface Segregation Principle
- D - Dependency Inversion Principle

Projektovati komponente sa velikom kohezijom (da se funkcionalnosti izmedju komponenti ne mesaju dok se podaci koje neka komponenta koristi u okviru svojim metoda treba da budu na njoj).
Na neku komponentu ne bi trebalo da uticu detalji implementacije neke druge komponente.
Razumeti kako ce komponente komunicirati.
Sloj poprecnih funkcionalnosti (bezbednosti, komunikacije, logovanje/operacioni menadzment) treba razdvojiti od logike aplikacije.
Primeniti kljucne principe arhitekturalnog stila baziranog na komponentama.

131. SRP (Single Responsibility Principle) *

Bavi se kohezijom na nivou komponenti. Njegovo pravilo je da svaka klasa treba da ima samo jednu odgovornost (tada klasa ima samo jedan razlog za promenu). Ukoliko klasa ima vise odgovornosti tada ima vise razloga za promenu. Promena jedne odgovornosti moze narusiti ili onemoguciti sposobnost klase da ispuni drugu odgovornost,

132. Open/Closed Principle (OCP) *

Softverski entiteti (klase, moduli, entitet) treba da budu otvoreni za prosirenja ali zatvoreni za modifikacije. Da bi se ovo implementiralo potrebno je da se koristi apstrakcija (primer server-klijent).Prosirenje predstavlja dodavanje novih funkcionalnosti a modifikacija promena postojecih. Modifikacija uzrokuje izmenu postojecih metoda i funkcionalnosti i direktno utica na uspesnost izvrsavanja testova cime je u mnogome ugrozena funkcionalnost softvera.

133. Liskov Substitution Principle *

Barbara Liskov kaze da bazni tipovi treba da budu zamenjivi njihovim podtipovima. Narusavanje LSP-a cesto uzrokuje narusavanje OCP-a (primer kocka i pravougaonik).

134. Interface Segregation Principle *

Klijenti ne treba da zavise od metoda koje ne koriste (sigurnosna vrata primer)

135. Dependency-Inversion Principle *

- Moduli viseg nivoa ne treba da zavise od modula nizeg nivoa i jedni i drugi bi trebalo da zavise od apstrakcije.
- Apstrakcija ne bi trebalo da zavisi od detalja nego bi detalji trebalo da zavise od apstrakcije.

Kada imamo distribuairane aplikacije koje poseduju objekte koji treba medjusobno da komuniciraju oni ne bi trebalo direktno da referenciraju jedan drugog nego da zavise od interfejsa koji jedan implementira a drugi koristi.